NORTHWESTERN UNIVERSITY

Contract Checking as an Effect

A DISSERTATION

SUBMITTED TO THE GRADUATE SCHOOL

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

for the degree

MASTER OF SCIENCE

Field of Computer Science

By

Anh Khoa Nguyen

EVANSTON, ILLINOIS

June 2026

ABSTRACT

Contract Checking as an Effect

Anh Khoa Nguyen

Design by Contracts is a development strategy focuses on placing contracts in function arguments, ensuring inputs and outputs are valid during runtime. Moreover, once a contract is broken, the runtime can report the developer which part of the code is responsible. In this thesis, we attemp to implement contracts using effect handlers, a unified approach to computational effects. We showed that contracts can be implemented as a wrapper around effect handlers. We also provided an implementation in OCaml, a language that has native support for effect handlers but not for contracts.

Acknowledgements

Text for acknowledgments.

Preface

This is the preface.

Table of Contents

List of Tables

List of Figures

CHAPTER 1

Introduction

Software contracts, or *behavioral contracts*, allow programmers to attach assertions to values, and functions. These assertions are runtime guarantees, on violation, an error is thrown, blaming the violating party. Contracts are most useful when placed on a function, enforcing the function's input and output values. While first-order functional contracts are useful, they are too simple. Higher-order functional contracts allows programmers to enforce contracts for higher-order functions. Dependent contracts are functional contracts allowing usage of the function argument during post-condition contract checking.

Recently, algebraic effect handlers has gained a large attraction, and is adopted in OCaml since version 5.0. Effect handlers open a new way to identify effects and handling them. Handlers are defined to handle one or many effects. A handler for an effect can stop the program or continue with some result. Handlers do not have state by default, but it can be implemented by applying the new states to the continuation closure, and let the underlying computation receives a state argument.

Combining the contract system and effect handlers is a new approach. Effectful Contract [7] demonstrates how both systems can be combined, providing contracts for effectful code. Contracts in their system remain a language feature. In this dissertation, instead of combining the effects and contracts, we ask ourselves whether contract checking itself can become an effect, and reuse the capabilities of handlers.

1.1. Contracts checking

Software contracts, despite their usefulness, are not natively support in many programming languages. Racket and D-lang are two languages that have native support for contracts. C++, or its 2026 version, has some basic support for contracts. Python has a proposal (PEP 316[1]), but it was deferred since 2003.

Racket implements contracts by code rewriting with macros. Moreover, protected values and functions are wrapped inside a native proxy object (chaperone). Native support allows Racket to implement higher-order contracts.

Without native support, the community have come up with strategies for implementing software contracts. Java's libraries (C4J[2] and Cofoja[3]), define contracts using Java's annotations and rewrite the bytecode during compilation to add contract checking code. Similar strategies are used by Rust (contracts[4]) and Golang (gocontracts[5]) libraries, where the code is rewritten to add pre and post condition code checking to a function.

Python has support for first-class functions, and *decorators*. Decorators syntactically wraps a function using another function while the original function name can still be referenced. This feature is used in many Python libraries (icontract[6], deal[7], etc.) to implement functional contracts.

Without native contract support, functional and dependent contracts are feasible. While higher-order contracts remains a challenge to implement.

[1] https://peps.python.org/pep-0316/
[2] https://c4j-team.github.io/C4J/index.html
[3] https://github.com/nhatminhle/cofoja
[4] https://crates.io/crates/contracts
[5] https://github.com/Parquery/gocontracts
[6] https://github.com/Parquery/icontract
[7] https://github.com/life4/deal

1.2. Contracts checking as effects

Contracts checking adds extra computation, when the checks pass, the program continues. When the checks fail, the program aborts. The behavior is similar to throwing an exception. Exception is one kind of side-effect. The similarities between exceptions and contracts checking are sufficient to make contracts checking an effect and use handlers to handle them.

Once contract effects are added, handlers need to be added. Correctly grouping the handlers can share the context of contracts. Sharing the context between contracts allows for optimization through contract caching, or cross-function call data sharing. Previously, cross-function call data is stored in a global storage [8], or using effects to provide "ghost" state [7].

1.3. Contributions

This thesis explores the "contracts checking as effects" approach to implementing software contracts. Through experiments, we show that this approach accomodates functional contracts, dependent contracts, and the challenging higher-order contracts. We also discovered that as effects, their contexts can be shared if they are under the same handler. Sharing contracts context opens new capabilities to optimizations and contract state sharing. This disseration shows the above idea by providing a compiler model between CPCF [1] and System F^{ϵ} [12]. We also provide an implementation in OCaml as a practical implementation.

The rest of the thesis is organized as follows: Chapter 2 and Chapter 3 introduces the two formal languages `Contracts` based on CPCF, and `Effects` based on System F^{ϵ}; Chapter 4 discusses a theoretical compiler from `Contracts` to `Effects` as a demonstration to "contract checking as effects"; Chapter 5 introduces the OCaml language and its native support for effect handlers; Chapter 6 provides the implementation of the theoretical compiler in OCaml using

PPX rewritter; Chapter 7 compares our implementation in OCaml to the Racket's contract system; Chapter 8 concludes our remarks.

CHAPTER 2

Software Contracts

Software Contracts place contracts, dynamic runtime assertions, into the program.

When the value is used, it is validated against the contracts and issue an error. The first proposal of this feature is the Effiel language [6], with the support for first-order function. Higher-order function and dependent function support [3], trace contracts [8], are useful functional addition on top of the base contracts.

2.1. Examples

2.2. Language `Contracts`

In this section, we reintroduce CPCF [1, 2] as `Contracts`.

Figure 2.1 describes the language of `Contracts`, with its type rules defined in 2.2 excluding common type rules in *PCF*. Values are basic number, booleans, and functions. Expressions are the usual forms of CPCF, with an extra expression $\mathsf{mon}_j^{k,l}(\kappa, e)$ that guards the expression e with a contract κ. A contract can be a flat contract $\mathsf{flat}(c)$, which can only be applied to a value, and verify the value against function c; a function contract $\kappa_1 \mapsto \kappa_2$, which guards the input arguments with a domain contract κ_1 and guards the output/return value with a range contract κ_2. Dependent contract $\kappa_1 \overset{d}{\mapsto} (\lambda x : \tau.\kappa_2)$ behaves similarly to a function contract but the range contract can use the input value. blame_p^l signals the location of the contract l and the party to blame p. Error is not a surface syntax of `Contracts`, it only happens during evaluation.

Reduction rules for `Contracts` is provided in Figure 2.3. A flat contract monitor expression reduces into an if expression checking if the value meets the contract predicate. Else, the entire program reduces to an error. A function contract monitor expression reduces into a function where its argument are wrapped with the domain contract, and the application result is wrapped with the range contract. Argument contract labels are changed for blame correctness. Dependent contract is similarly reduces like a function contract, but the range contract has the input value wrapped up with the domain contract with different labels set. The labels set used in dependent contract is by the indy semantics, which has been proven about its correctness [2].

This thesis introduces two languages. To effectively distinguish the languages used, we have chosen different typeset styling for both languages. `Contracts` will be typeset in blue san − serif with $\longrightarrow$ is the reduction arrow, Γ is the typing context.

Basic Types	o	::=	num | bool
Types	τ	::=	o | $\tau_1 \to \tau_2$ | con(τ)
Contracts	κ	::=	flat(e) | $\kappa_1 \mapsto \kappa_2$ | $\kappa_1 \overset{d}{\mapsto} (\lambda x : \tau.\kappa_2)$
Expressions	e	::=	0, 1, ... | tt | ff | x | $\lambda x : \tau$.e | $\mu x : \tau$.e
		|	$e_1 + e_2$ | $e_1 - e_2$ | $e_1 \wedge e_2$ | $e_1 \vee e_2$ | $e_1\ e_2$
		|	if e_1 then e_2 else e_3 | zero?(e)
		|	$\text{mon}_j^{k,l}(\kappa, e)$ | blame_p^l
Values	v	::=	0, 1, ... | tt | ff | λx.e
Contexts	E	::=	$\square$ | E + e | v + E | E $\wedge$ e | v $\wedge$ E | E $\vee$ e | v $\vee$ E
		|	E e | v E | if E then e else e | zero?(E)
		|	$\text{mon}_j^{k,l}(\kappa, E)$

Figure 2.1. Syntax of Language `Contracts`

$$\textsc{Type-C-Error}\ \frac{}{\Gamma \vdash \mathsf{blame}^{l}_{p} : \tau} \qquad \textsc{Type-C-Mon}\ \frac{\Gamma \vdash \kappa : \mathsf{con}(\tau) \qquad \Gamma \vdash e : \tau}{\Gamma \vdash \mathsf{mon}^{k,l}_{j}(\kappa, e) : \tau} \qquad \textsc{Type-C-FlatContract}\ \frac{\Gamma \vdash e : o \to \mathsf{bool}}{\Gamma \vdash \mathsf{flat}(e) : \mathsf{con}(o)}$$

$$\textsc{Type-C-FuncContract}\ \frac{\Gamma \vdash \kappa_1 : \mathsf{con}(\tau_1) \qquad \Gamma \vdash \kappa_2 : \mathsf{con}(\tau_2)}{\Gamma \vdash \kappa_1 \mapsto \kappa_2 : \mathsf{con}(\tau_1 \to \tau_2)} \qquad \textsc{Type-C-DepContract}\ \frac{\Gamma \vdash \kappa_1 : \mathsf{con}(\tau_1) \qquad \Gamma, x : \tau_1 \vdash \kappa_2 : \mathsf{con}(\tau_2)}{\Gamma \vdash \kappa_1 \overset{d}{\mapsto} (\lambda x : \tau_1.\kappa_2) : \mathsf{con}(\tau_1 \to \tau_2)}$$

Figure 2.2. Type Rules for `Contracts`

$$\textsc{Step-C-If-True}\ \frac{}{\mathsf{if\ tt\ then}\ e_1\ \mathsf{else}\ e_2 \longrightarrow e_1} \qquad \textsc{Step-C-If-False}\ \frac{}{\mathsf{if\ ff\ then}\ e_1\ \mathsf{else}\ e_2 \longrightarrow e_2} \qquad \textsc{Step-C-App}\ \frac{}{(\lambda x.e)\ v \longrightarrow \{v/x\}e}$$

$$\textsc{Step-C-Fix}\ \frac{}{\mu x.e \longrightarrow \{\mu x.e/x\}e} \qquad \textsc{Step-C-Mon-Flat}\ \frac{}{\mathsf{mon}^{k,l}_{j}(\mathsf{flat}(e), v) \longrightarrow \mathsf{if}\ e\ v\ \mathsf{then}\ v\ \mathsf{else}\ \mathsf{blame}^{j}_{k}}$$

$$\textsc{Step-C-FuncContract}\ \frac{}{\mathsf{mon}^{k,l}_{j}(\kappa_1 \mapsto \kappa_2, v) \longrightarrow \lambda x.\mathsf{mon}^{k,l}_{j}(\kappa_2, v\ (\mathsf{mon}^{l,k}_{j}(\kappa_1, x)))}$$

$$\textsc{Step-C-DepContract}\ \frac{}{\mathsf{mon}^{k,l}_{j}(\kappa_1 \overset{d}{\to} (\lambda x.\kappa_2), v) \longrightarrow \lambda x.\mathsf{mon}^{k,l}_{j}(\{\mathsf{mon}^{l,j}_{j}(\kappa_1, x)/x\}\kappa_2, v\ (\mathsf{mon}^{l,k}_{j}(\kappa_1, x)))}$$

$$\textsc{Step-C-Context}\ \frac{e_1 \longrightarrow e_2}{E[e_1] \longrightarrow E[e_2]} \qquad \textsc{Step-C-Err}\ \frac{E \neq \square}{E[\mathsf{blame}^{l}_{p}] \longrightarrow \mathsf{blame}^{l}_{p}} \qquad \textsc{Step-C-Refl}\ \frac{}{e \longrightarrow^{*} e} \qquad \textsc{Step-C-Trans}\ \frac{e \longrightarrow e_1 \qquad e_1 \longrightarrow^{*} e_2}{e \longrightarrow^{*} e_2}$$

Figure 2.3. Reduction Rules for `Contracts`

CHAPTER 3

Algebraic Effect Handlers

Algebraic Effects is the mathematical model for computational effects first introduced in [9] as an extension to the λ-calculus. After some time, Effect Handlers are introduced in [10] as a new addition to programming languages. Algebraic Effects and Handlers allows programmers to write programs with complex control flows. Functionalities such as exception handling, asynchronousity, I/O operations, and others are proven to be implement-able using Algebraic Effect Handlers. In this thesis, we use the term Algebraic Effect Handlers, Effect Handlers, and Handlers interchangably.

Algebraic Effect Handlers adapts the use of Call-By-Push-Value [5] to separate value types, and computation types. This separation helps typing the computation together with possible effects, whereas value types are pure. This formal model usually provides reduction rules for its semantics.

In another spectrum, Effect Handlers are modeled by a combination of row types on top of System F_ω, named System F^ϵ[12]. The type rules are adjusted to accompany with a list of effects for an expression. This formal model usually provides evaluation contexts for its semantics.

3.1. Examples

3.2. Language `Effects`

In this section, we introduce `Effects`, a formal model of the Algebraic Effect Handlers as a model language for our target language for the compiler. Previously mentioned, we have two

main choices for the formal language, (i) in the flavour of fine-grain call-by-value or (ii) in the flavour of System F^{ϵ}. We have chosen to adapt the (ii) flavour into the formal language due to its evaluation context semantics, which is helpful for our proofs.

We adopt and extend the language of System F^{ϵ} such that it can support all expressions available to `Contracts`, including number type, binary operations, and if expression. We also introduce a new expression blame p blaming the party p.

The language is polymorphic, with higher-kinded type system where effect is treated as a different kind from basic type, where in traditional effect type system, they are expressed as computational type. Effects are represented using row types, each row is indexed by a label. The environment of effect labels Σ maps each labels to operations.

Programmers can define operations to be handled by rows of h, consisting of operation names, and their behaviors. A handler is also a value, and can be applied with a function (handler h f) which a shorthand to a handle expression with f applied to unit. In general, handle h e is used to run e with effect handler h. During evaluation of e, an operation can be invoked by applying perform op σ to a value. If op is declared in any handler of the current context, then the operation is handled using the closest handler. Because we define `Effects` well-typedness as no unhandled effects, there is no such case that an operation is undeclared once the program is well-typed.

A basic value, booleans and numbers, has no effect. A function $\sigma_1 \to \epsilon\sigma_2$ receiving a type σ_1 producing a σ_2 value can invoke effects in ϵ. A handler $h = \{op_1 \to f_1, ..., op_n \to f_n\}$ must unify all operation types into one, allowing effect label of these operations, and other effects allowed by the environment. Because there is no type unification, therefore, these operations must return the same type.

Type judgement $\Delta \vdash e : \sigma \mid \epsilon$ keeps a list of effects ϵ available to the expression e being typed with type σ.

Following [12], we adopt the *dot notation* to decompose and compose evaluation contexts, as defined in Figure 3.1. We would write $v \cdot \text{handle } h \cdot E \cdot e$ as a shorthand for v (handle h ($E[e]$)).

This thesis introduces two languages. To effectively distinguish the languages used, we have chosen different typeset styling for both languages. `Effects` will be typeset in red serif with $\twoheadrightarrow$ is the reduction arrow, and Δ is the typing context.

Context Composition	$E \cdot e$	$\doteq$	$E[e]$
Argument Evaluation	$\square\ e \cdot E$	$\doteq$	$E\ e$
Function Evaluation	$v\ \square \cdot E$	$\doteq$	$v \cdot E \doteq v\ E$
Handler Application	$\text{handle } h\ \square \cdot E$	$\doteq$	$\text{handle } h\ \cdot E \doteq \text{handle } h\ E$

Figure 3.1. `Effects` Evaluation Context Dot Notation

Types	σ	$::=$	$\text{num} \mid \text{bool} \mid \alpha^k \mid c^k\ \sigma \ldots \sigma \mid \sigma \to \epsilon\sigma \mid \forall\alpha^k.\sigma$
Kinds	k	$::=$	$* \mid k \to k \mid \text{eff} \mid \text{lab}$
Expressions	e	$::=$	$v \mid e_1\ e_2 \mid e[\sigma] \mid \text{handle } h\ e$
		$\mid$	$e_1 + e_2 \mid e_1 - e_2 \mid e_1 \wedge e_2 \mid e_1 \vee e_2$
		$\mid$	$\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \mid \text{zero?}(e)$
		$\mid$	$\text{blame } p$
Values	v	$::=$	$x \mid 0, 1, \ldots, N \mid \text{tt} \mid \text{ff} \mid \lambda x : \sigma.e$
		$\mid$	$\Lambda\alpha^k.v \mid \text{handler } h \mid \text{perform op } \overline{\sigma}$
Handlers	h	$::=$	$\{\text{op}_1 \to f_1, \ldots, \text{op}_n \to f_n\}$
Eval Contexts	F	$::=$	$\square \mid F\ e \mid v\ F \mid F[\sigma]$
	E	$::=$	$\square \mid E\ e \mid v\ E \mid E[\sigma] \mid \text{handle } h\ E$

Figure 3.2. Syntax of Language `Effects`

$$\textsc{Type-E-X-Val}\ \frac{x : \sigma \in \Delta}{\Delta \vdash_{val} x : \sigma} \qquad \textsc{Type-E-Lambda}\ \frac{\Delta, x : \sigma_1 \vdash e : \sigma_2 \mid \epsilon}{\Delta \vdash_{val} \lambda x : \sigma_1.e : \sigma_1 \to \epsilon\sigma_2} \qquad \textsc{Type-E-Val}\ \frac{\Delta \vdash_{val} v : \sigma}{\Delta \vdash v : \sigma \mid \epsilon}$$

$$\textsc{Type-E-TypeLambda}\ \frac{\Delta \vdash_{val} v : \sigma \qquad k \neq lab}{\Delta \vdash_{val} \Lambda\alpha^k.v : \forall\alpha^k.\sigma} \qquad \textsc{Type-E-If}\ \frac{\Delta \vdash e_1 : bool \mid \epsilon \qquad \Delta \vdash e_2 : \sigma \mid \epsilon \qquad \Delta \vdash e_3 : \sigma \mid \epsilon}{\Delta \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \sigma \mid \epsilon}$$

$$\textsc{Type-E-BinOp-Nat}\ \frac{\Delta \vdash e_1 : nat \mid \epsilon \qquad \Delta \vdash e_2 : nat \mid \epsilon}{\Delta \vdash e_1 \oplus_{nat} e_2 : nat \mid \epsilon} \qquad \textsc{Type-E-BinOp-Bool}\ \frac{\Delta \vdash e_1 : bool \mid \epsilon \qquad \Delta \vdash e_2 : bool \mid \epsilon}{\Delta \vdash e_1 \oplus_{bool} e_2 : bool \mid \epsilon}$$

$$\textsc{Type-E-Zero}\ \frac{\Delta \vdash e : nat \mid \epsilon}{\Delta \vdash \text{zero? } e : bool \mid \epsilon} \qquad \textsc{Type-E-App}\ \frac{\Delta \vdash e_1 : \sigma_1 \to \sigma \mid \epsilon \qquad \Delta \vdash e_2 : \sigma_1 \mid \epsilon}{\Delta \vdash e_1\, e_2 : \sigma \mid \epsilon}$$

$$\textsc{Type-E-TypeApp}\ \frac{\Delta \vdash e : \forall\alpha^k.\sigma_1 \mid \epsilon \qquad \vdash_{wf} \sigma : k}{\Delta \vdash e[\sigma] : \sigma_1[\alpha := \sigma] \mid \epsilon} \qquad \textsc{Type-E-Perform}\ \frac{op : \forall\overline{\alpha}.\sigma_1 \to \sigma_2 \in \Sigma(l) \qquad \overline{\alpha} \notin ftv(\Delta)}{\Delta \vdash_{val} \text{perform op } \overline{\sigma} : \sigma_1[\overline{\alpha} := \overline{\sigma}] \to \langle l|\epsilon\rangle\, \sigma_2[\overline{\alpha} := \overline{\sigma}]}$$

$$\textsc{Type-E-Ops}\ \frac{op_i : \forall\overline{\alpha}.\sigma_1 \to \sigma_2 \in \Sigma(l) \qquad \overline{\alpha} \notin ftv(\epsilon\,\sigma) \qquad \Delta \vdash_{val} f_i : \forall\overline{\alpha}.\sigma_1 \to (\sigma_2 \to \epsilon\sigma) \to \epsilon\sigma}{\Delta \vdash_{ops} \{op_1 \to f_1, \ldots, op_n \to f_n\} : \sigma \mid l \mid \epsilon}$$

$$\textsc{Type-E-Handler}\ \frac{\Delta \vdash_{ops} h : \sigma \mid l \mid \epsilon}{\Delta \vdash_{val} \text{handler } h : (() \to \langle l|\epsilon\rangle\, \sigma) \to \sigma} \qquad \textsc{Type-E-Handle}\ \frac{\Delta \vdash_{ops} h : \sigma \mid l \mid \epsilon \qquad \Delta \vdash e : \sigma \mid \langle l|\epsilon\rangle}{\Delta \vdash \text{handle } h\, e : \sigma \mid \epsilon}$$

$$\textsc{Type-E-Blame}\ \frac{}{\Delta \vdash \text{blame } p : \sigma \mid \epsilon}$$

Figure 3.3. Type Rules for System `Effects`

$$\textsc{Step-E-If-True}\quad \frac{}{\text{if tt then } e_1 \text{ else } e_2 \rightsquigarrow e_1} \qquad \textsc{Step-E-If-False}\quad \frac{}{\text{if ff then } e_1 \text{ else } e_2 \rightsquigarrow e_2} \qquad \textsc{Step-E-App}\quad \frac{}{(\lambda x : \sigma.e)\ v \rightsquigarrow e[x := v]}$$

$$\textsc{Step-E-TApp}\quad \frac{}{(\Lambda\alpha^{k}.v)[\sigma] \rightsquigarrow v[\alpha := \sigma]} \qquad \textsc{Step-E-Handler}\quad \frac{}{(\text{handler } h)\ v \rightsquigarrow \text{handle } h \cdot v\ ()} \qquad \textsc{Step-E-Return}\quad \frac{}{\text{handle } h \cdot v \rightsquigarrow v}$$

$$\textsc{Step-E-Perform}\quad \frac{\begin{array}{c}\text{op} \notin \text{bop}(E) \wedge (\text{op} \to f) \in h \\ \text{op} : \forall\overline{\alpha}.\sigma_1 \to \sigma_2 \in \Sigma(l) \qquad k = \lambda x : \sigma_2[\overline{\alpha} := \overline{\sigma}].\text{handle } h \cdot E \cdot x\end{array}}{\text{handle } h \cdot E \cdot \text{perform op } \overline{\sigma}\ v \rightsquigarrow f[\overline{\sigma}]\ v\ k} \qquad \textsc{Step-E-Context}\quad \frac{e_1 \rightsquigarrow e_2}{E[e_1] \rightsquigarrow E[e_2]}$$

$$\textsc{Step-E-Blame}\quad \frac{E \neq \square}{E[\text{blame } k] \rightsquigarrow \text{blame } k} \qquad \textsc{Step-E-Refl}\quad \frac{}{e \rightsquigarrow^{*} e} \qquad \textsc{Step-E-Trans}\quad \frac{e \rightsquigarrow e_1 \qquad e_1 \rightsquigarrow^{*} e_2}{e \rightsquigarrow^{*} e_2}$$

Figure 3.4. Reduction Rules for System `Effects`

3.3. Soundness and Correctness

Our algebraic effects handler system has type soundness by progress (Theorem 3.3.1) and preservation (Theorem 3.3.2). Our effect system is also safe, i.e., all effects are handled or specified in the judgement. To express safety of effects, we first define (in Figure 3.5), $\lceil E \rceil^{l}$ extracts all labels in the context; and bop(E) extracts all op in the context. Lemma 3.3.3 and Lemma 3.3.4 express effect safety.

Theorem 3.3.1 (`Effects` Progress). If $\cdot \vdash e_1 : \sigma \mid \langle\rangle$, then either e_1 is a value, or there exists an e_2 such that $e_1 \rightsquigarrow e_2$.

Theorem 3.3.2 (`Effects` Preservation). If $\cdot \vdash e_1 : \sigma \mid \langle\rangle$ and $e_1 \rightsquigarrow e_2$, then $\cdot \vdash e_2 : \sigma \mid \langle\rangle$.

Lemma 3.3.3 (Well-typed operations are handled). If $\cdot \vdash E[\text{perform op } \overline{\sigma}\ v] : \sigma \mid \langle\rangle$, then E has the form $E_1 \cdot \text{handler } h \cdot E_2$ with $\text{op} \notin \text{bop}(E_2)$ and $\text{op} \to f \in h$.

$$\lceil \mathrm{E} \rceil^{l} = \lceil \mathrm{F}_0 \cdot \mathrm{handle}\ \mathrm{h}_1 \cdot \mathrm{F}_1 \cdot \cdots \cdot \mathrm{handle}\ \mathrm{h}_n \cdot \mathrm{F}_n \rceil^{l} = \langle l_n, \ldots, l_1 \rangle \text{ iff } \mathrm{h}_i : \Sigma(l_i)$$

$$\begin{aligned}
\mathrm{bop}(\square) &= \varnothing \\
\mathrm{bop}(\mathrm{E}\ \mathrm{e}) &= \mathrm{bop}(\mathrm{E}) \\
\mathrm{bop}(\mathrm{v}\ \mathrm{E}) &= \mathrm{bop}(\mathrm{E}) \\
\mathrm{bop}(\mathrm{handle}\ \mathrm{h}\ \mathrm{E}) &= \mathrm{bop}(\mathrm{E}) \cup \{\mathrm{op} \mid (\mathrm{op} \to \mathrm{f}) \in \mathrm{h}\}
\end{aligned}$$

Figure 3.5. Metafunction for context in `Effects`.

Lemma 3.3.4 (Effects types are meaningful). If $\cdot \vdash \mathrm{E}[\mathrm{perform}\ \mathrm{op}\ \overline{\sigma}\ \mathrm{v}] : \sigma \mid \epsilon$ with $\mathrm{op} \notin \mathrm{bop}(\mathrm{E})$, then $\mathrm{op} \in \Sigma(l)$ and $l \in \epsilon$.

CHAPTER 4

Theoretical Compiler

In this chapter, we describe a theoretical compiler from `Contracts` to `Effects`. The idea to the compilation is simple, with special handling for dependent contract. A monitor of flat contract will be converted into performing an effect. A monitor of function contract will be recursively expanded similarly to the reduction until only flat contracts exist. Monitor of dependent contract is compiled similarly to the flat contract, with a slight difference for dependent argument.

The compilation described above only add in effect performing, without their handler component. Without a suitable handler and using handle expression, the program cannot run. First, we define that each contract can be converted into a handler. The handler accepts a non-function value, and check it against the contract predicate. If the value passes the predicate, the program continues (using the handler continuation) with the value. Else the program steps to a error, which stops the program with the party responsible for the blame.

With a handler defined, we need to use it with a handle expression to ensure effects performed are handled. The easiest approach is to apply the handle right where the effect is performed with its corresponding handler. This does not give any advantage to basic if-else compilation. Handlers can stay inside the same context if they are used together. We suggest grouping them together, this allows contracts to access the same context information.

Grouping them together is easy, and we can place a handle at the whole program. However, this approach fails for dependent contract checking. Dependent contract κ_d checking happens

$$\mathsf{mon}_j^{k,l}(\mathsf{flat}(\lambda x : \mathsf{num.zero?}(x{-}10)), 9)$$
$$\Downarrow$$

$$\begin{array}{l} h \triangleq \{\mathsf{check} \rightarrow \\ \quad \Lambda\alpha.\lambda x : \alpha.\lambda \mathsf{kont} : \alpha \rightarrow \langle\rangle\alpha.\mathsf{if}\ (\lambda x : \mathsf{num.zero?}(x - 10))\ x\ \mathsf{then\ kont\ x\ else\ blame\ k}\} \\ \\ \mathsf{perform\ check\ num\ 9} \end{array}$$

Figure 4.1. Example compilation of flat contract

while executing another contract κ_c. If κ_c is running inside a handler, which is true in our described compilation, then the handler is moved into the continuation, leaving κ_c running without a handle context. To resolve this, we have to handle κ_d using a handler of all possible effects that might arise during execution of κ_d. This loses the general context shared across all other contracts, but guarantees all effects are handled.

The next section illustrates many examples aiming to give an intuition to the reader before the later formal section, where a compilation model between `Contracts` to `Effects` is presented.

4.1. Examples

We first examine the flat contract case in Figure 4.1. A flat contract compiles into performing effect operation check, a unique effect name generated for this contract. The handler h defined with only the operation check, and not applied immediately to the compiled expression. Handler h will be kept going forward the compilation process.

A function contract is then straightforward by recursively compile the argument and the application result into `Effects`. Figure 4.2 demonstrates the unrolling of function contract into single flat contracts and compiling them into their corresponding operation invocation. Similarly to flat contract, the group of handlers are not used immediately.

$$\mathrm{mon}_j^{k,l}((\mathrm{flat}(\kappa_1) \mapsto \mathrm{flat}(\kappa_2)) \mapsto \mathrm{flat}(\kappa_3), f)$$
$$\Downarrow$$
$$\lambda g.\mathrm{mon}_j^{k,l}(\mathrm{flat}(\kappa_3), f\ (\mathrm{mon}_j^{l,k}(\mathrm{flat}(\kappa_1) \mapsto \mathrm{flat}(\kappa_2), g)))$$
$$\Downarrow$$
$$\lambda g.\mathrm{mon}_j^{k,l}(\mathrm{flat}(k_3), f\ (\lambda x.\mathrm{mon}_j^{l,k}(\mathrm{flat}(\kappa_2), g\ (\mathrm{mon}_j^{k,l}(\mathrm{flat}(\kappa_1), x)))))$$
$$\Downarrow$$

$$h_1 \triangleq \{\mathrm{check}_{\kappa_1} \to \ldots \text{ blame } k\}$$
$$h_2 \triangleq \{\mathrm{check}_{\kappa_2} \to \ldots \text{ blame } l\}$$
$$h_3 \triangleq \{\mathrm{check}_{\kappa_3} \to \ldots \text{ blame } k\}$$

$$e_1 \triangleq \text{perform } \mathrm{check}_{\kappa_1}\ \tau_1\ x$$
$$e_2 \triangleq \text{perform } \mathrm{check}_{\kappa_2}\ \tau_2\ (g\ e_1)$$
$$e_3 \triangleq \text{perform } \mathrm{check}_{\kappa_3}\ \tau_3\ (f\ (\lambda x.e_2))$$

$$\lambda g.e_3$$

Figure 4.2. Example compilation of higher-order contract

A dependent contract compiles similarly to a function contract. However, because the handler scope during checking the post contract is pushed into the continuation, the dependent contract must be wrapped in its own handler scope. Figure 4.3 describe the procedure for a *basic type* argument. Because the argument is a basic type, there is only one operation in the handler h_3. If the argument is a function, then h_3 should capture all possible effects, collectable after compiling recursively with known contract.

$$\mathsf{mon}^{k,l}_j(\mathsf{flat}(\kappa_1) \overset{d}{\mapsto} (\lambda x.\mathsf{flat}(k_2)), g)$$
$$\Downarrow$$
$$\lambda x.\mathsf{mon}^{k,l}_j(\{\mathsf{mon}^{l,j}_j(\mathsf{flat}(\kappa_1), x)/x\}\mathsf{flat}(\kappa_2), g\ (\mathsf{mon}^{l,k}_j(\mathsf{flat}(\kappa_1), x)))$$
$$\Downarrow$$

$$\begin{aligned}
h_1 &\triangleq \{\mathsf{check}_{\kappa_1} \to \ldots \text{ blame } l\} \\
h_2 &\triangleq \{\mathsf{check}_{\kappa_2} \to \\
&\quad \Lambda\alpha.\lambda y : \alpha.\lambda \mathsf{kont} : \alpha \to \langle\rangle\alpha.\text{if } \kappa_2[x := x_{dep}] \text{ then kont } y \text{ else blame } k \\
h_3 &\triangleq \{\mathsf{check}'_{\kappa_1} \to \ldots \text{ blame } l\}
\end{aligned}$$

$$\begin{aligned}
x_{arg} &\triangleq \text{perform check}_{\kappa_1}\ \tau_1\ x \\
x_{dep} &\triangleq \text{handle } h_3\ (\text{perform check}'_{\kappa_1}\ \tau_1\ x) \\
e_1 &\triangleq \text{perform check}_{\kappa_2}\ \tau_2\ (g\ x_{arg})
\end{aligned}$$

$$\lambda x.e_1$$

Figure 4.3. Example compilation of dependent contract

4.2. Type-Directed Compilation

The compilation relies heavily on the type of argument. Therefore, we provide a type-directed compilation from `Contracts` to `Effects`. Compilation relation $\Gamma \vdash e : \tau \twoheadrightarrow h, e'$ takes an expression e of type τ with context Γ and compiles into e' with a handler h.

The compilation is recursive, types are used to form well-typed `Effects` expressions. We define the following theorems for our compiler: Type Preservation, Effect Safe Compiler, Observable Equivalance, and Blame Preservation. Type Preservation Theorem states that a well-typed program in `Contracts` compiles into a well-typed program in `Effects` typed using the same basic type and has some effects. Effect Safe Compiler Theorem makes sure that all (contract) effects are handled-able using the compiler outputed handler. Observable Equivalance Theorem

and Blame Preservation Theorem both ensure the same semantic behavior is preserved whether the program executed correctly or a blame occured.

Because `Effects` has different types from `Contracts`, we define $\mathtt{T}$ as the type conversion from Γ to Δ. The conversion is simple because `Contracts` only has basic type, so we map to the basic type of `Effects`.

In Figure 4.5, we provide some interesting compilation rules, other expressions left out should be trivial. Types used in `Effects` that copied from `Contracts`context are assumed to be converted.

Environment Translation

$$\mathtt{T}(\cdot) \triangleq \cdot$$
$$\mathtt{T}(\Gamma, \mathtt{x} : \tau) \triangleq \mathtt{T}(\Gamma), \mathtt{x} : \tau^*$$

Type Translation $(\tau \mapsto \tau^*)$

$$\mathtt{b}^* \triangleq \mathtt{b} \qquad \text{(where b is a basic type like num)}$$
$$(\tau_1 \to \tau_2)^* \triangleq \tau_1^* \to \langle\rangle\tau_2^*$$

Figure 4.4. Type and Environment Translation from `Contracts` to `Effects`
Basic types are converted into basic types in kind system. Function types are computation without any effects. Contract type has no equivalent type in `Effects`. Context type translation $\mathtt{T}$ converts all variables in `Contracts` into their equivalent types in `Effects`.

Theorem 4.2.1 (Type Preservation). If $\Gamma \vdash \mathtt{e} : \tau \twoheadrightarrow \mathtt{h}, \mathtt{e}'$ then $\Delta \vdash \mathtt{e}' : \tau^* \mid \epsilon$ for some ϵ and $\mathtt{T}(\Gamma) \subseteq \Delta$

Theorem 4.2.2 (Effect Safe Compiler). If $\Gamma \vdash \mathtt{e} : \tau \twoheadrightarrow \mathtt{h}, \mathtt{e}'$ and $\mathtt{T}(\Gamma) \subseteq \Delta$ then $\Delta \vdash \mathtt{handle\ h\ e}' : \tau^* \mid \langle\rangle$

Theorem 4.2.3 (Observable Equivalance). If $\mathtt{e} \longrightarrow^* \mathtt{v}$ and $\cdot \vdash \mathtt{e} : \tau \twoheadrightarrow \mathtt{h}, \mathtt{e}'$ then there exists a $\mathtt{v}'$ such that $\mathtt{handle\ h\ e}' \rightsquigarrow^* \mathtt{v}'$

Theorem 4.2.4 (Blame Preservation). If $\mathsf{e} \longrightarrow^* \mathsf{blame}^{\mathsf{l}}_{\mathsf{p}}$ and $\cdot \vdash \mathsf{e} : \tau \twoheadrightarrow \mathsf{h}, \mathsf{e}'$ then $\mathsf{handle\ h\ e}' \rightsquigarrow^* \mathsf{blame\ p}$

$$\textsc{Compile-Var}\ \frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau \twoheadrightarrow \{\}, x} \qquad \textsc{Compile-Val}\ \frac{}{\Gamma \vdash v : \tau \twoheadrightarrow \{\}, v} \qquad \textsc{Compile-Lambda}\ \frac{\Gamma, x : \tau_1 \vdash e : \tau_2 \twoheadrightarrow h, e'}{\Gamma \vdash \lambda x : \tau_1.e : \tau_1 \to \tau_2 \twoheadrightarrow h, \lambda x : \tau_1^*.e'}$$

$$\textsc{Compile-If}\ \frac{\Gamma \vdash e_1 : \mathsf{bool} \twoheadrightarrow h_1, e_1' \qquad \Gamma \vdash e_2 : \tau \twoheadrightarrow h_2, e_2' \qquad \Gamma \vdash e_3 : \tau \twoheadrightarrow h_3, e_3'}{\Gamma \vdash \mathsf{if}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3 : \tau \twoheadrightarrow h_1 \cup h_2 \cup h_3, \mathsf{if}\ e_1'\ \mathsf{then}\ e_2'\ \mathsf{else}\ e_3'}$$

$$\textsc{Compile-BinOp-Nat}\ \frac{\Gamma \vdash e_1 : \mathsf{nat} \twoheadrightarrow h_1, e_1' \qquad \Gamma \vdash e_2 : \mathsf{nat} \twoheadrightarrow h_2, e_2'}{\Gamma \vdash e_1 \oplus_{\mathsf{nat}} e_2 : \mathsf{nat} \twoheadrightarrow h_1 \cup h_2, e_1' \oplus_{\mathsf{nat}} e_2'}$$

$$\textsc{Compile-BinOp-Bool}\ \frac{\Gamma \vdash e_1 : \mathsf{bool} \twoheadrightarrow h_1, e_1' \qquad \Gamma \vdash e_2 : \mathsf{bool} \twoheadrightarrow h_2, e_2'}{\Gamma \vdash e_1 \oplus_{\mathsf{bool}} e_2 : \mathsf{bool} \twoheadrightarrow h_1 \cup h_2, e_1' \oplus_{\mathsf{bool}} e_2'}$$

$$\textsc{Compile-App}\ \frac{\Gamma \vdash e_1 : \tau_1 \to \tau \twoheadrightarrow h_1, e_1' \qquad \Gamma \vdash e_2 : \tau_1 \twoheadrightarrow h_2, e_2'}{\Gamma \vdash e_1\ e_2 : \tau \twoheadrightarrow h_1 \cup h_2, e_1'\ e_2'}$$

$$\textsc{Compile-MonFlat}\ \frac{\Gamma \vdash e : \tau \twoheadrightarrow h, e' \qquad \mathsf{check}_\kappa \notin h \qquad h_\kappa \triangleq \{\mathsf{check}_\kappa \to f\} \qquad f \triangleq \Lambda\alpha.\lambda x : \alpha.\lambda \mathsf{kont} : \alpha \to \langle\rangle\alpha.\mathsf{if}\ \kappa\ x\ \mathsf{then}\ \mathsf{kont}\ x\ \mathsf{else}\ \mathsf{blame}\ k}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\mathsf{flat}(\kappa), e) : \tau \twoheadrightarrow h \cup h_\kappa, \mathsf{perform}\ \mathsf{check}_\kappa\ \tau^*\ e'}$$

$$\textsc{Compile-MonFunc}\ \frac{x \notin \Gamma \qquad \Gamma, x : \tau_1 \vdash \mathsf{mon}_j^{k,l}(\kappa_2, e\ (\mathsf{mon}_j^{l,k}(\kappa_1, x))) : \tau_2 \twoheadrightarrow h, e'}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e) : \tau_1 \to \tau_2 \twoheadrightarrow h, \lambda x : \tau_1^*.e'}$$

$$\textsc{Compile-MonDep}\ \frac{y, \delta_y \notin \Gamma \qquad \Gamma, y : \tau_1 \vdash \mathsf{mon}_j^{l,j}(\kappa_1, y) : \tau_1 \twoheadrightarrow h_1, e_y \qquad \Gamma, y : \tau_1 \vdash \mathsf{mon}_j^{k,l}(\{\delta_y/x\}\kappa_2, e\ (\mathsf{mon}_j^{l,k}(\kappa_1, y))) : \tau_2 \twoheadrightarrow h_2, e' \qquad h_2' = h_2[\delta_y := \mathtt{wrap}(h_1, e_y)]}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \overset{d}{\mapsto} (\lambda x.\kappa_2), e) : \tau_1 \to \tau_2 \twoheadrightarrow h_1 \cup h_2', \lambda y : \tau_1^*.e'}$$

Helper function

$$\mathtt{wrap}(h, e) \triangleq \mathsf{handle}\ h\ e \quad (e \neq \lambda x.e')$$

$$\mathtt{wrap}(h, \lambda x.e) \triangleq \lambda x.\mathtt{wrap}(h, e)$$

Figure 4.5. Type-Directed Compilation from `Contracts` to `Effects`
Original types in `Contracts` is converted into their equivalent types in `Effects` after compilation. When compiling dependent contracts, a separate variable is used to store the dependent argument then replaced on top the compiled expression.

CHAPTER 5

OCaml

OCaml is a programming language in the flavour of ML-style family. It was first designed as a research language, and gradually became a mainstream programming language. The language is now open sourced on Github[1].

5.1. Effect Handlers support

OCaml added support for Effect Handlers since OCaml 5.0[2], released in 2022. This feature was added to Multicore OCaml following the work of [11], which is later merged into main branch (OCaml 5.0). Before the official support of Effect Handlers, [4] implemented Effect Handlers by embedding Eff into OCaml using Module Functors.

In OCaml, effects must be defined by extending the Effect type with variants. These variants is also the effect's name (or label in `Effects`). The effect can receive none or at most one argument, and must return an effect type. To invoke a certain effect, `Effect.perform` is called with the effect variant (and argument if exists).

Handlers are defined using multiple syntaxes, as effects can be pattern matched. A basic syntax is by matching the computation in `try computation () with`, illustrated in Listing 1. This is a standard syntax for exception handling in OCaml, extended to support effects by matching an `effect (EffectName param), kontinuation`. An alternative syntax is by `match computation () with` with the same idea.

[1] https://github.com/ocaml/ocaml
[2] https://ocaml.org/manual/5.4/effects.html

```
type _ Effect.t += Incr: int -> int t
let compute () = perform (Incr 0)

try compute () with
| effect (Incr n), k -> continue k (n + 1)
```

Listing 1. OCaml effects handler extended by exception handler

Using the exception handling syntax is short but it does not provide the full Effect Handler capability. For example, an effect handler with state cannot be expressed in this syntax. Therefore, OCaml official syntax for handling effects is by using `Effect.Deep.match_with` for deep handlers, and `Effect.Shallow.continue_with` for shallow handlers. These functions provides the same handler structure as in `Contracts`, with return handler, and effect handlers. This handler also support exception handlers as well. If return handlers and exception handlers are ignored, then OCaml provides `Effect.Deep.try_with` that accepts only effect handlers. Listing 2 shows an effect handler with state.

5.2. Software Contracts support

Software Contracts are not supported in OCaml. Previously, [13] implemented software contracts into the OCaml language. However, it has never been merged into the official OCaml, and the work is staled in the `contract-3.1.1` branch[3].

5.3. Metaprogramming in OCaml

OCaml allows sophisticated macros for metaprogramming. In fact, OCaml allows any program to change the source code before compiling. OCaml provides a toolset for parsing and modifying the OCaml at the AST level, instead of writing a customized parser/modifier from

[3] https://github.com/ocaml/ocaml/tree/contracts-3.11

```
type _ Effect.t +=
  | Get : int Effect.t
  | Set : int -> unit Effect.t

let run_state (f : unit -> 'a) (init : int) : 'a =
  let handler = {
    retc = (fun x _state -> x);
    exnc = raise;
    effc = (fun (type a) (eff : a Effect.t) ->
      match eff with
      | Get -> Some (fun (k : (a, int -> 'b) continuation) ->
          fun state -> continue k state state)
      | Set s -> Some (fun (k : (a, int -> 'b) continuation) ->
          fun _state -> continue k () s)
      | _ -> None)
  } in
  Effect.Deep.match_with f () handler init

let program () =
  let v = Effect.perform Get in
  Effect.perform (Set (v + 10));
  Effect.perform Get

let result = run_state program 5
```

Listing 2. OCaml effects handler with state by `Effect.Deep.match_with`

scratch. The utility is often referred to as the *PPX*, or the Preprocessor of OCaml. With a given Preprocessor ppx library, the OCaml compiler can be specified to run the preprocessor before compilation. This entire process is supported also by Dune, the OCaml project manager.

5.3.1. Attributes

When writing a preprocessor, the developer would want to focus on rewriting a certain part of the code. Therefore, as a suggestion, preprocessor should rewrite code annotated with their defined attribute(s). Attributes[4] are OCaml metadata appended to the AST, by the syntax of

[4] https://ocaml.org/manual/5.4/attributes.html

```
let [@rename y] x = 1
```

Listing 3. Example attribute

`[@attribute-id]` or `[@@attribute-id]` (depending on their position on the code). Attributes can also carry at most one structured payload. An example attribute, provided in Listing 3, that can annotate a preprocessor to rename variable to the variable in the payload.

CHAPTER 6

Implementation

We have identified OCaml Effect Handlers are similar to `Effects`. Therefore we can implement a contract system op top of OCaml and use preprocessor to rewrite the AST accordingly to the theoretical compiler. We first define a contract system in OCaml in the form of attributes. These annotations are defined similarly to the contract κ in `Contracts`.

6.1. Examples

In Listing 4 is the code annotated with contracts, including contracts for values, functions, higher-order function, and dependent contracts.

Annotations are built using an attribute `@contract` attached to the `let` expression. This attribute is only valid when the `let` expression is a value, or a function, and must be annotated with its types.

In the example, `x` is constrained to bigger than 10. If the provided value of `x` breaks the contract, an exception is thrown as part of blame. The exception is checked lazily only when `x` is being used. `f1`, `f2`, `f3` are functions, and they are annotated with function contract, dependent contract, and higher-order dependent contract respectively. Their semantics are identical to the semantics higher-order contracts.

```
let any _ = true
let bigger_10 num = num > 10
let bigger_20 num = num > 20
let total_bigger_50 x v = (x + v) > 50

let [@contract : bigger_10] x : int = 9

let [@contract : bigger_10 -> bigger_20]
  f1 (x : int) : int = x

let [@contract : bigger_10 -> total_bigger_50 dep]
  f2 (x : int) : int = x

let pass g v =
  (* by the contract in f3, g must be bigger_10 -> bigger_20 *)
  let vv = g (v - 20) in
  vv > 30

let [@contract : any -> ((bigger_10 -> bigger_20) -> pass dep)]
  f3 (x : int) (g : int -> int) : int = g x
```

Listing 4. Example contracts annotations in OCaml

6.2. Annotations

OCaml attributes have a certain structure[1]. We chose to implement the annotations as a type expression in OCaml of the form $\tau \rightarrow \tau$. This choice limits what we can put in as the predicates. Therefore, we only allow predicates pre-defined as a function or available/imported at the variable annotated.

We define the following annotations that are similar to different types of contract: flat, function, dependent.

A flat contract is applied for a single value ρ, annotated using a single name. This name is the function used to check the value.

[1]Attributes must follow the attribute-payload structure as defined in https://ocaml.org/manual/5.4/attributes.html

```
1 exception Blame of string
```

Listing 5. Blame exception

A function is composabled by many flat contracts in the form $\rho \rightarrow ... \rightarrow \rho$. In OCaml a function can receive many arguments. These arguments can be values or functions. Part of the OCaml type signature, which we utilized for contract annotation, parenthesis can be used to separate between the main function contract, and contracts for arguments that are functions.

A dependent contract is strictly a function receiving a single argument $\rho \rightarrow \rho\ dep$. This choice is made to keep the semantics similar to `Contracts`. Alternatively, we can choose to implement the dependent contract as checking with all arguments, $\rho \rightarrow .. \rightarrow \rho\ dep$. For this thesis, we keep the contract system simple and allow only for single argument.

6.3. Preprocessor Rewriting

We implemented a rewritter as a preprocessor using `ppxlib`[2]. The rewriter walks the AST of OCaml, finds and modifies valid `@contract` annotated `let` expression.

The rewritter also updates the variable into a function receiving three extra arguments `pos,neg,cloc` for blame labels. `pos` is the positive label, which is the blamed party for a flat contract. `neg` is the negative label. `cloc` is the contract location used for dependent contract blame assignment. Blame is designed as an exception.

In the theoretical compiler, a list of handlers is accumulated. A careful reader may notice that most handlers have the same structure, with two differences, the predicate, and the blame label. Therefore, as an optimization, we design a single effect type accepting a *tuple of three values*: the

[2] https://ocaml-ppx.github.io/ppxlib/ppxlib/index.html

```
type _ Effect.t +=
  | Flat : (string * ('a -> bool) * 'a) -> 'a Effect.t

let rec contract_checking : type a b.
  a Effect.t
  -> ((a, b) Effect.Deep.continuation -> b) option
  = fun eff ->
  let open Effect.Deep in
  match eff with
  | Flat (label, check, arg) ->
      Some (fun k ->
        if check arg
        then continue k arg
        else discontinue k (Blame label))
  | _ -> None
```

Listing 6. Contract Effect

blame label, the predicate function, and the value to check. We also define unify handler for this effect which has the contract checking logic. These are illustrated in Listing 6.

This is an optimization when contracts do not need shared state. We can augment this with a state provided to the `contract_checking` and apply predicate with the state. At the moment, the lack the semantics about sharing states between contracts, so the current implementation works only as a demonstration.

6.3.1. Flat contract rewriting

Flat contracts rewrite, in Listing 7, by converting the value to performing the contract effect `Flat` with the appropriate arguments. Blame labels are provided as argument to the orginal function such that when used, correct blame label is applied.

```
(* original program *)
let [@contract : predicate] var : v_type = value

(* rewritten program *)
let var pos neg cloc =
  Effect.perform (Contract.Flat (pos, predicate, value));
```

Listing 7. Flat contract rewrite

6.3.2. Functional contract rewriting

With flat contracts defined, a functional contract rewrite can be broken down into contracts for its arguments and its return value, as illustrated in Listing 8. Therefore a function receiving 3 arguments, will be rewritten into 4 sub-contracts, 3 for its arguments, and 1 for its return value. An exception for this rule is when the function has a function in one or more of its arguments. Then the rewrite must effectively rewrite the argument contract into a function to rewrite recursively. When rewriting into a function, argument names are generated. Blame labels for arguments are "flipped" according to the `Contracts` semantics.

6.3.3. Dependent contract rewriting

Dependent contract has a predicate that receives the argument value. Therefore, we build up the correct contract predicate for the output contract. The value must be passed in with a different labels set from the value used, which we only need to create a new random variable. Not only that, because the dependent argument runs inside a handler context, it lacks a handler because the contract effect handler is pushed into the continuation. Therefore, we must wrap it into a handler, as in the theoretical compiler model. Then we provide this to the post-condition predicate as argument. The process is recursive, if the dependent argument also has a dependent contract the

```
(* original program *)
let [@contract : xpred -> ypred -> zpred -> outpred]
  f1 (x : x_type) (y : y_type) (z : z_type) : f_type = body

let [@contract : (g_inpred -> g_outpred) -> outpred]
  f2 (g : g_intype -> g_outtype) : f_type = body


(* rewritten program *)
let f1 pos neg cloc (x : x_type) (y : y_type) (z : z_type) : f_type =
  let [@contract : xpred] x : x_type = x in
  let x = x neg pos cloc in
  let [@contract : ypred] y : y_type = y in
  let y = y neg pos cloc in
  let [@contract : zpred] z : z_type = z in
  let z = z neg pos cloc in
  let [@contract : outpred] ret : f_type = body in
  ret pos neg cloc

let f2 pos neg cloc (g : g_intype -> g_outtype) : f_type =
  let [@contract : g_inpred -> g_outpred]
    g (g_p1 : g_intype) : g_outtype = g g_p1 in
  let g = g neg pos cloc in
  let [@contract : outpred] ret : f_type = body in
  ret pos neg cloc
```

Listing 8. Functional contract rewrite

dependent argument is also wrapped inside a handler. An example rewrite is provided in Listing 9.

6.3.4. Handling Contract Effects

Using the defined `contract_checking` user can wrap all possible code path to handle all contract effects. User should be aware of functions with contract effects called outside the handler. Therefore, we suggest wrapping the whole program under `contract_checking`. This is equivalent to Effect Safety Compiler Theorem.

```
(* original program *)
let [@contract :
    (xpred * (gp1_p1_pred -> gp1_pred) -> gpred as 'dep) -> outpred as 'dep]
  f (x : x_type) (g : g_type) : f_type = body


(* rewritten program *)
let f pos neg cloc (x : x_type) (g : g_type) : f_type =
  let [@contract : xpred] x : x_type = x in
  let x_dep = x neg cloc cloc in
  let x = x neg pos cloc in

  let [@contract : (gp1_p1_pred -> gp1_pred) -> gpred as 'dep]
    g (g_p1 : gp1_intype) : g_outtype = g g_p1 in
  let g_dep =
    fun p1 -> Contract.run_with_effects (fun () -> g neg cloc cloc p1) in
  let g = g neg pos cloc in

  let outpred_dep = outpred x_dep g_dep in
  let [@contract : outpred_dep] ret : f_type = body in
  ret pos neg cloc

```

Listing 9. Dependent contract rewrite

6.3.5. Blame labels assignment

All uses of the variable must now be appended with the three parameters `pos`, `neg`, and `cloc`. It is by practice to use the module name hosting the variable as the positive label, the module name of usesite as the negative label, and the variable name itself as the contract label. However, during the limitation of preprocessor having limited scoping functionality, we hereby define a simpler strategy for blame assingment. There are three cases that we have to resolve for blame assignment.

- Function's argument

This blame assignment follows the semantics of `Contracts`, where positive and negative labels are ”swapped” with each other on function input. Because function arguments are bounded inside a function, the generated code applies the labels of the parent function in the correct order. This process happens for dependent contract as well, where the dependent argument is applied with the correct labels from its parent function.

- Global variable

  Once a protected functions/values are used in the user code, they must be applied with the correct labels, else the blame report confuses the programmer. There are 2 main cases here, (i) the function is defined used in the same module, (ii) the function is defined and used in two different modules. We examine the two cases and demonstrate how should the blame labels be assigned.

  In the same module scenario, the positive label is the module and the function. The negative label is the code usage, which is the same module. Because our implementation does not transform recursive functions, the only place in the same module that can use a protected function/value is other function, or in annonymous scope. If the protected function is used inside another function, the negative label is the name of the module and the hosting function. If it is being used in an annonymous scope, the negative label is the name of the module.

  In different modules scenario, the positive label remains the module and the function. The negative label is the code usage. Similarly, we can use the module name with the function name if unannoymous.

In both of these scenarios, the contract location label is the same as the positive label.

- Local variable

  Because a contract can be annotated in any let expression, it is possible to annotate a local variable with contract. In this scenario, the positive label is the module, and the hosting function. The negative label is where the variable is being used, which is highly likely the function itself. Therefore all three labels are the same.

CHAPTER 7

Comparison to Racket's contract system

Previously, we described an implementation contract system in OCaml. Our implementation relies heavily on the preprocessor rewriting engine to wrap a value or function with its contract. Guarded values are rewritten into a function to provide blame labels and checked when all labels are provided. In this section we provided a few insight into the Racket's contract system, then compare it to our approach in OCaml. We will see that implementing contract system in OCaml is a challenge compared to Racket due to OCaml's restriction.

7.1. Racket's contract system

The Racket language provides two main functionalities that makes implementing contract system easier: hygenic macros, and chaperone. The contract system of Racket[1] is implemented using (hygenic) macros to modify the sourcecode. Afterwards, guarded values are wrapped inside a chaperone. Racket macros allows implementors to manipulate the AST in a more sensible manner during compile time. It replaces the need for a preprocessor, and target specific. Chaperone on the other hand, provides a special run-time object manipulation that is useful for contract system implementation.

A chaperone is an *invisble proxy* over a value. When the value is used, the chaperone intercepts and perform some predefined actions. In the case of software contract, the contract

[1] https://github.com/racket/racket/tree/master/racket/collects/racket/contract

checking is the chaperone action. A chaperone cannot do anything, for such is allowed by impersonator. The chaperone action can throw an exception, which is used to signal a contract violation. Chaperone cannot be used to implement flat contracts, but it can be used to implement function contracts using `chaperone-procedure`. This function takes a function and a wrapper function, which is applied for every argument, and return value(s). The wrapper function body should follows the (higher-order and dependent) functional contract semantics.

Chaperone is also lazy. A chaperone action on a vector or structure only runs when accessing the element (vector) or member (structure). This can be thought of as an optimization to lazily check until the value is actually needed.

7.2. Comparison

OCaml preprocessor and Racket macros are both used for the sole purpose of rewriting the AST to add contract checking functionalities to the sourcecode, eventhough OCaml provides a more generic system (preprocessor can modify any part of the AST). In both implementations, the guarded value must be wrapped. Racket's chaperone has support for function wrappers, lifting heavy and complex logic into a single function invocation (by `chaperone-procedure` and its family). OCaml does not have such functionalities, nor can it be implemented on top of the language (judging by Racket's choice of implementation[2]). Therefore, functions are modified to wrapped each arguments and the body (return value).

Through our observation, we identify two main aspects that our implementation is greatly different from Racket's contract system: (i) Contract checking, and (ii) Blame assignment.

[2]Chaperone is implemented as part of the language of Racket

7.2.1. Contract checking

Our current implementation limits to values (as a whole) and functions. In Racket's contract system, contracts for vectors and structures is also implemented using chaperone to guard an element access (vector), or member access (structure). A chaperone enforces contract lazily, only when an element is accessed or on member access. Our implementation, however, perform contract checking for arrays and structs as an entity bringing in more runtime overhead.

Racket's chaperone can perform multi-shot contract checking, compared to our implementation where contract is checked only one time. For instance, a chaperone of vector guards its element across mutation; our contract system limits to one time contract checking. Vector, tuple and structure contracts in our implementation lacks this fine-grain control. This limitation is because OCaml's lack of chaperone-like mechanism.

One exception. Structures in OCaml, records, can be made into their functional versions using the preprocessor *ppx_fields_conv*.[3] Using the functions generated, we can implement our contract system to append contracts to these functions and provide contracts for structures. However, the original record dot notation is still unprotected.

7.2.2. Blame assignment

Blame assignment in Racket is handled by chaperone. By storing blame labels inside the "proxy" object, the protected values can be used without changing their invocation.

In our implementation, all protected values is converted into a function accepting three additional arguments for labels. Therefore, we need to search for all usages, and provide the correct blame labels. It becomes a hurdle when function signatures must also be modified.

[3] https://github.com/janestreet/ppx_fields_conv

CHAPTER 8

Conclusion

Our work is the first to use effects for contract checking in a contract system. We introduced a theoretical compiler that can transform a language with native support for contracts into a language with native support for effect handlers. The compiler is proven sound, and semantics are preserved. As a proof of concept we have implemented a minimal contract library in OCaml 5.0+, which has native support for effect handlers. Our library uses attributes to annotate contract similarly to how a contract system would, and transform the AST during compilation using a preprocessor rewriter.

We gave a comparison of our OCaml implementation to Racket's contract system. Through the comparison, our implementation is similar to Racket's. However, Racket's use of chaperone provides better contract checking logic than ours. Moreover, our implementation is limited to one-shot contract checking, and would not be able to handle mutation.

8.1. Future Works

This thesis experiments with effects as contract checking, and has proven that normal contract semantics are represent-able using effect handlers alone. We have not discussed extended models of software contracts such as trace contracts, and effectful contracts, etc... Effectful contracts system integrates contracts for effects. Our work and the effectful contracts system both have effects, investigating whether their contract system can be compiled into a language supporting effect handlers is a future extension. It also follows that trace contracts are implemented

if the previous work is feasible since effectful contracts system can be used to implement trace contracts.

References

[1] Dimoulas, C., and Felleisen, M. On contract satisfaction in a higher-order world. *ACM Trans. Program. Lang. Syst. 33*, 5 (Nov. 2011).

[2] Dimoulas, C., Tobin-Hochstadt, S., and Felleisen, M. Complete monitors for behavioral contracts. In *European Symposium on Programming* (2012), Springer, pp. 214–233.

[3] Findler, R. B., and Felleisen, M. Contracts for higher-order functions. In *Proceedings of the seventh ACM SIGPLAN international conference on Functional programming* (2002), pp. 48–59.

[4] Kiselyov, O., and Sivaramakrishnan, K. Eff directly in ocaml. *arXiv preprint arXiv:1812.11664* (2018).

[5] Levy, P. B. *Call-by-push-value*. PhD thesis, 2013.

[6] Meyer, B. The eiffel programming language. *See http://www. eiffel. com* (1992).

[7] Moy, C., Dimoulas, C., and Felleisen, M. Effectful software contracts. *Proc. ACM Program. Lang. 8*, POPL (Jan. 2024).

[8] MOY, C., and FELLEISEN, M. Trace contracts. *Journal of Functional Programming 33* (2023), e14.

[9] Plotkin, G., and Power, J. Algebraic operations and generic effects. *Applied categorical structures 11*, 1 (2003), 69–94.

[10] Plotkin, G. D., and Pretnar, M. Handling algebraic effects. *Logical methods in computer science 9* (2013).

[11] Sivaramakrishnan, K., Dolan, S., White, L., Kelly, T., Jaffer, S., and Madhavapeddy, A. Retrofitting effect handlers onto ocaml. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (2021), pp. 206–221.

[12] Xie, N., Brachthäuser, J. I., Hillerström, D., Schuster, P., and Leijen, D. Effect handlers, evidently. *Proceedings of the ACM on Programming Languages 4*, ICFP (2020), 1–29.

[13] Xu, D. N. Dynamic contract checking for ocaml, 2014.